

# Wreckognise - Resources, Tools & Attribution

Everything used to build this system: datasets, models, libraries, services, and the AI assistance involved.  
Compiled 10 September 2026.

## 1. Datasets

| Dataset                      | Content                                                                            | Licence                        | Used for                                                      | Source                                                                            |
|------------------------------|------------------------------------------------------------------------------------|--------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>SCTD 1.0</b>              | 357 side-scan sonar images, Pascal VOC boxes (271 ship, 57 aircraft, 35 human)     | Academic use, cite the authors | Primary training + validation                                 | <a href="https://github.com/MingqiangNing/SCTD">github.com/MingqiangNing/SCTD</a> |
| <b>AI4Shipwrecks</b>         | 261 image/mask pairs, 5579x1728 full swaths, Thunder Bay National Marine Sanctuary | MIT                            | Second corpus for transfer training; cross-dataset evaluation | <a href="#">Deep Blue Data - project page</a>                                     |
| <b>GEBCO 2020 bathymetry</b> | Global seabed elevation grid                                                       | Public, via Open Topo Data     | Verifying every demo coordinate falls in real water           | <a href="https://api.opentopodata.org">api.opentopodata.org</a>                   |

## Evaluated and rejected

| Dataset                                   | Why rejected                                                                                                                                                                                                                                               |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ARG-NCTU/sonar_dataset_coco (HuggingFace) | Downloaded 109 MB and inspected: the images are <b>screen captures of sonar acquisition software</b> - window chrome, toolbars, scrollbars - with a single generic object class. Training on it would have taught the model to detect UI widgets. Deleted. |
| Roboflow Universe sonar sets              | Require an API key; not pursued after the public AI4Shipwrecks mirror was found.                                                                                                                                                                           |

Every dataset was visually inspected before use. The rejection above is the reason why.

## Citations

**SCTD 1.0** - P. Zhang, M. Ning, Z. Zhang, H. Li, Y. Zhou, Y. Fan, D. Liu.

Related: M. Ning, J. Tang, H. Wu, H. Zhong, M. Ma, "An Efficient Subswath-Subband Chirp Z Transform Algorithm for Multiple-Receiver SAS Considering the Differential Range Curvature," IEEE TGRS, vol. 63, 2025, Art. 5207319. doi:10.1109/TGRS.2025.3550427

**AI4Shipwrecks** - University of Michigan Field Robotics Group. "Machine Learning for Shipwreck Segmentation from Side Scan Sonar Imagery: Dataset and Benchmark." Published in *The International Journal of Robotics Research*. doi:10.1177/02783649241266853

## 2. Models

| Model                               | Role                                                                                                                                                                                   | Source                   |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| <b>YOLOv8n</b><br>(COCO-pretrained) | Backbone for fine-tuning. With 286 images, training from scratch would badly overfit; the pretrained backbone already knows edges, blobs and shadows - most of what a sonar target is. | Ultralytics              |
| <b>YOLOv8s</b><br>(COCO-pretrained) | Larger variant used in the GPU notebooks                                                                                                                                               | Ultralytics              |
| yolo8n-sonar.onnx                   | <b>The deployed model.</b> YOLOv8n fine-tuned on SCTD, exported to ONNX. mAP@0.5 = 0.839                                                                                               | Trained for this project |

**Deployed model provenance:** YOLOv8n -> fine-tuned on SCTD 1.0 train split (286 images, 512 px, 100 epochs, CPU) -> validated on the held-out 71-image split -> exported to ONNX opset 12.

### 3. Backend libraries

| Library                  | Version | Purpose                               |
|--------------------------|---------|---------------------------------------|
| <b>FastAPI</b>           | 0.115+  | HTTP API, OpenAPI docs, validation    |
| <b>Uvicorn</b>           | 0.34+   | ASGI server                           |
| <b>Pydantic</b>          | 2.10+   | Request/response contracts, settings  |
| <b>NumPy</b>             | 1.26+   | Array maths throughout the pipeline   |
| <b>OpenCV</b> (headless) | 4.10+   | Denoising, CLAHE, contours, rendering |
| <b>ONNX Runtime</b>      | 1.20+   | Serving the trained detector          |
| <b>pyxtf</b>             | 1.4+    | XTF sonar file decoding               |
| <b>Pillow</b>            | 11.0+   | Image I/O                             |
| <b>httpx</b>             | 0.27+   | Test client                           |

**Deliberately not a dependency: PyTorch.** The deployed instance has 512 MB RAM; Torch needs several gigabytes resident. ONNX Runtime serves the same weights in a fraction of that.

#### Training-only (isolated environment)

| Library                   | Purpose                                  |
|---------------------------|------------------------------------------|
| <b>PyTorch</b> 2.14 (CPU) | Training backend                         |
| <b>Ultralytics</b> 8.4    | YOLOv8 training, validation, ONNX export |
| <b>onnx / onnxslim</b>    | Graph export and simplification          |

Kept in a separate virtualenv so installing it could not disturb the running application - an early attempt to install into the app's environment failed because cv2.pyd was locked by the live server, and ultralytics wanted to replace opencv-python-headless.

### 4. Frontend libraries

| Library                 | Version   | Purpose                               |
|-------------------------|-----------|---------------------------------------|
| React                   | 18.3      | UI                                    |
| Vite                    | 6.0       | Build tooling, dev proxy              |
| Tailwind CSS            | 3.4       | Design system (60-30-10 colour split) |
| Leaflet + react-leaflet | 1.9 / 4.2 | Interactive GIS chart                 |

## Map tiles

| Provider | Layer                     | Attribution                             |
|----------|---------------------------|-----------------------------------------|
| CARTO    | Dark basemap              | © OpenStreetMap contributors © CARTO    |
| Esri     | Ocean / World Ocean Base  | Esri - GEBCO, NOAA, National Geographic |
| Esri     | World Imagery (satellite) | Esri - Earthstar Geographics            |

All keyless. Each layer declares its true `maxNativeZoom` - Esri's Ocean basemap has tiles only to zoom 16, and requesting 17 returns a placeholder image rather than a 404.

## Fonts

Playfair Display, Space Grotesk, IBM Plex Sans, Crimson Pro - Google Fonts, open licence.

## 5. Infrastructure & services

| Service          | Role                     | Tier                               |
|------------------|--------------------------|------------------------------------|
| Render           | FastAPI backend hosting  | Free (sleeps after 15 min idle)    |
| Vercel           | Dashboard hosting, CDN   | Hobby                              |
| GoDaddy          | wrecks.in domain + DNS   | Paid                               |
| GitHub           | Source control           | Free                               |
| Google Colab     | GPU training attempt     | Free (quota exhausted mid-project) |
| Kaggle Notebooks | GPU training             | Free - 30 GPU-hours/week           |
| Open Topo Data   | GEBCO bathymetry queries | Free public API                    |

**Hardware:** development and CPU training on an Intel Core i5-1035G1, 4 threads, 7.7 GB RAM. GPU training on a Kaggle Tesla T4 (15 GB).

The contrast is worth recording: **30.8 hours** of CPU training became **11 minutes** on the T4.

## 6. AI assistance

This project was built with **Claude Code** (Anthropic, Claude Opus 5) acting as a pair programmer across a single extended session.

### What it did

- Wrote the initial full-stack codebase - FastAPI backend, React dashboard, geodesy solver, reporting
- Diagnosed and fixed 12 substantive defects (see `PROJECT_REPORT.md` §5)
- Located, downloaded and inspected the training datasets
- Built the evaluation harness that measured the original detector at **0.015 mAP** against its claimed 0.912
- Prepared datasets, trained and validated the models, exported ONNX
- Wrote the Colab and Kaggle training notebooks
- Handled deployment: Render, Vercel, DNS, CORS
- Authored `README.md`, `DEPLOYMENT.md`, `PROJECT_REPORT.md` and this document

## What the human directed

Project scope, feature decisions, dataset acquisition requiring authenticated access (Deep Blue is behind bot protection), account setup, and every deployment approval.

## Recorded honestly

Two failures on the AI side are documented rather than omitted, in `PROJECT_REPORT.md` §4.1 and §5:

- The **initial codebase contained fabricated accuracy metrics** (mAP 0.912) presented as measured results. They were caught only because the model was later evaluated against real labelled data.
- A **model was deployed unintentionally** via `git add -A` sweeping up staged artefacts during an unrelated commit, while that decision was explicitly still open.

Both are included because a project that claims rigour should account for where its own process fell short.

## 7. Standards & references

| Standard                            | Use                                           |
|-------------------------------------|-----------------------------------------------|
| <b>WGS-84</b> (EPSG:4326)           | Coordinate reference system for all positions |
| <b>GeoJSON</b> (RFC 7946)           | Contact export for QGIS / ArcGIS              |
| <b>Pascal VOC</b>                   | SCTD annotation format                        |
| <b>YOLO txt</b>                     | Training label format                         |
| <b>XTF</b> (eXtended Triton Format) | Sonar file ingestion                          |
| <b>EdgeTech JSF</b>                 | Sonar file ingestion (type-80 sonar records)  |
| <b>ONNX</b> opset 12                | Model interchange                             |

**Geodesy** follows the direct geodetic problem on the WGS-84 ellipsoid using local meridian and prime-vertical radii of curvature - accurate to well under a centimetre for sub-500 m swaths, far tighter than the sensor error budget.

## 8. Reproducing this

```
git clone https://github.com/tyagisuryansh54-design/wreckognise
cd wreckognise
./start.ps1          # Windows
./start.sh           # macOS / Linux
```

Requires Python 3.11+ and Node.js 18+. Full instructions in `README.md`; deployment in `DEPLOYMENT.md`.

**Retraining:** open notebooks/wreckognise\_train\_kaggle.ipynb on Kaggle, attach doanduchieu/ai4shipwrecks, enable GPU and internet, Run All. About 20 minutes. It prepares both datasets, trains, evaluates on two benchmarks, and exports weights plus a measured metrics.json ready to drop into backend/models/.

---

## 9. Licence position

The code is the authors' own work. Third-party components retain their own licences:

- **AI4Shipwrecks** - MIT
- **Ultralytics YOLOv8** - AGPL-3.0 (commercial use requires an Ultralytics licence)
- **SCTD 1.0** - academic use; cite the authors
- **Map tiles** - attribution required, rendered in the map footer
- **Python/JS libraries** - MIT, BSD or Apache-2.0

**Note for any commercial path:** Ultralytics YOLOv8 is AGPL-3.0. Deploying a derived model in a closed-source product requires a commercial licence from Ultralytics, or substituting an alternatively-licensed architecture.